September 5, 2026 at 20:22

1. Introduction. Exercise 7.2.2.1–129 asks how many of MacMahon's coloured-triangle patterns are symmetrical. A pattern has *strong* symmetry if some rotation or reflection of the hexagon leaves it alone apart from a permutation of the colours, and *weak* symmetry if some rotation or reflection preserves its colour patches—the boundaries between differently coloured regions—with no condition on the colours at all.

The answer works out both counts by hand, splitting each case into six types and reporting a number for each. This program checks those numbers. It cannot check them one type at a time, because the types are a bookkeeping device of the answer's own; what it can do is count the whole of each symmetric family and compare with the total the six types predict. That turns out to be a sharp test: the multiplier between the two is fixed by the group, so each identity either comes out exactly or not at all.

Every identity comes out exactly but one. The weak left-right count is 28,280,400 where the answer's numbers predict 27,047,856, and the difference is $96 \cdot 12839$—a whole number of classes. The last three starred sections chase that down: the family is cut into pieces along the answer's own case analysis, and one piece, worth 12,839 essentially distinct patterns, turns out to be missing from the answer's six types. So the grand total there should be 294,457 rather than 281,618. Everything else stands.

One thing to notice before starting. Answer 126 pins the border of the hexagon to a single colour, and finds 11,853,792 solutions. Answer 129 says it uses "the options of answer 126", but it reports 68,024,064 solutions for a subfamily of them, which is more than answer 126 has altogether. So the border is free here, and every model below leaves it alone.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 4 ⟩
⟨ Functions 5 ⟩
func main( ) {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. The modes run from cheap to expensive. *syms* prints the table of the twelve symmetries; *base* reproduces answer 126; *strongref* sweeps every reflection against every colour involution to show that strong reflection symmetry cannot happen; *strong* and *tile* take a few minutes; *special* a couple of seconds; and *weak* takes anywhere from a minute and a half to thirteen hours, depending on which group is asked for.

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "syms",
    "syms, base, strongref, strong, tile, weak, or special")
group := flag.String("g", "rot", "for weak: rot, top, left, both, s3, or rot3")
dump := flag.Bool("dump", false, "print the problem instead of solving it")
fixedMask := flag.Int("fixed", -1,
    "for weak-g left: which of the four fixed triangles carry a solid tile")
conf := flag.String("conf", "",
    "for weak-g left: which two orbits carry the solid tiles, as i,j")
check := flag.Bool("check", false,
    "rebuild the patches of every solution and take a census")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
switch *mode {
case "syms":
    ⟨ Print the twelve symmetries 15 ⟩
case "base":
    ⟨ Reproduce answer 126 23 ⟩
case "strongref":
    ⟨ Ask whether a strong symmetry is possible 33 ⟩
case "strong":
    ⟨ Count the strongly symmetric placements 41 ⟩
case "tile":
    ⟨ Count the ones that tile the plane 44 ⟩
case "special":
    ⟨ Count the special placements 56 ⟩
case "weak":
    ⟨ Count the weakly symmetric placements 53 ⟩
default:
    fmt.Println("unknown mode")
}
```

This code is used in section 1.

4. The hexagon. Answer 124 names an upward-pointing triangle (x, y) and puts the downward-pointing one immediately to its right at $(x, y)'$. The hexagon of (59) has four rows holding 5, 7, 7 and 5 triangles.

⟨ Declarations 4 ⟩ ≡

```
type tri struct {
    x, y int
    down bool
}
```

This code is also defined in sections 12, 17, 35, 38, 42, 63, 67, 68, 71.

This code is used in section 1.

5. ⟨ Functions 5 ⟩ ≡

```
func (t tri) name() string {
    if t.down {
        return fmt.Sprintf("%d%d'", t.x, t.y)
    }
    return fmt.Sprintf("%d%d", t.x, t.y)
}
```

This code is also defined in sections 6, 7, 8, 9, 10, 11, 13, 14, 21, 22, 24, 25, 27, 28, 36, 37, 39, 43, 45, 46, 47, 57, 60, 62, 69, 70, 72, 73, 83.

This code is used in section 1.

6. Each triangle has three edges, which answer 126 writes in the order horizontal, then the two slants. Two triangles that touch name their common edge the same way, which is the whole point of the coordinate system.

⟨ Functions 5 ⟩ +≡

```
func (t tri) edges() [3]string {
    if t.down {
        return [3]string{
            fmt.Sprintf("-%d%d", t.x, t.y + 1),
            fmt.Sprintf("/%d%d", t.x + 1, t.y),
            fmt.Sprintf(`\%d%d`, t.x, t.y),
        }
    }
    return [3]string{
        fmt.Sprintf("-%d%d", t.x, t.y),
        fmt.Sprintf("/%d%d", t.x, t.y),
        fmt.Sprintf(`\%d%d`, t.x, t.y),
    }
}
```

7. ⟨ Functions 5 ⟩ +≡

```
func hexagon( ) []tri {
    rows := []struct { upLo, upHi, dnLo, dnHi int }{
        {2, 3, 1, 3}, {1, 3, 0, 3}, {0, 3, 0, 2}, {0, 2, 0, 1},
    }
    var out []tri
    for y, r := range rows {
        for x := r.upLo; x ≤ r.upHi; x++ {
            out = append(out, tri{x, y, false})
        }
        for x := r.dnLo; x ≤ r.dnHi; x++ {
            out = append(out, tri{x, y, true})
        }
    }
    return out
}
```

8. The tiles. A tile is a triangle with a colour on each edge, and turning it about its centre does not make a new tile. So the $4^3 = 64$ coloured triples fall into 24 tiles, named by the least of the three rotations of the triple.

⟨ Functions 5 ⟩ +≡

```
func canon(c [3]byte) string {
    best := string(c[:])
    for i := 1; i < 3; i++ {
        if r := string([]byte{c[i % 3], c[(i + 1) % 3], c[(i + 2) % 3]}); r < best {
            best = r
        }
    }
    return best
}
```

9. ⟨ Functions 5 ⟩ +≡

```
func tiles() []string {
    seen := map[string]bool{}
    for a := byte('a'); a ≤ 'd'; a++ {
        for b := byte('a'); b ≤ 'd'; b++ {
            for c := byte('a'); c ≤ 'd'; c++ {
                seen[canon([3]byte{a, b, c})] = true
            }
        }
    }
    var out []string
    for k := range seen {
        out = append(out, k)
    }
    sort.Strings(out)
    return out
}
```

10. The twelve symmetries. Answer 124 remarks that a triangle is a triple of barycentric coordinates summing to 2 (pointing up) or 1 (pointing down), and that the symmetries of the hexagon are the six permutations of those coordinates together with an optional flip $c \mapsto 1 - c$. Shifting the origin to the middle of the hexagon:

$$(x, y) \mapsto (x - 1,\ y - 1,\ 4 - x - y), \qquad (x, y)' \mapsto (x - 1,\ y - 1,\ 3 - x - y).$$

⟨ Functions 5 ⟩ +≡

```
func bary(t tri) [3]int {
    if t.down {
        return [3]int{t.x − 1, t.y − 1, 3 − t.x − t.y}
    }
    return [3]int{t.x − 1, t.y − 1, 4 − t.x − t.y}
}
```

11. ⟨ Functions 5 ⟩ +≡

```
func unbary(b [3]int) tri {
    return tri{b[0] + 1, b[1] + 1, b[0] + b[1] + b[2] ≡ 1}
}
```

12. The three coordinates are also the three edge directions, so a symmetry permutes a triangle's edges exactly as it permutes the coordinates. That single fact is all the models need to know about geometry.

⟨ Declarations 4 ⟩ +≡

```
type sym struct {
    perm [3]int
    flip bool
    name string
}
```

13. ⟨ Functions 5 ⟩ +≡

```
func (s sym) apply(t tri) tri {
    b := bary(t)
    var c [3]int
    for i := 0; i < 3; i++ {
        c[s.perm[i]] = b[i]
    }
    if s.flip {
        for i := range c {
            c[i] = 1 − c[i]
        }
    }
    return unbary(c)
}
```

14. An odd permutation reverses orientation, so it is a reflection; the flip does not, being the half-turn in disguise.

⟨ Functions 5 ⟩ +≡

```
func symmetries() []sym {
    perms := [][3]int{{0,1,2},{0,2,1},{1,0,2},{1,2,0},{2,0,1},{2,1,0}}
    var out []sym
    for _, p := range perms {
        odd := 0
        for i := 0; i < 3; i++ {
            for j := i + 1; j < 3; j++ {
                if p[i] > p[j] {
                    odd++
                }
            }
        }
        kind := "rotation"
        if odd % 2 == 1 {
            kind = "reflection"
        }
        for _, f := range []bool { false, true }{
            out = append(out, sym{p, f, fmt.Sprintf("%v flip=%v %s", p, f, kind)})
        }
    }
    return out
}
```

15. The answer says that top-bottom reflection changes every triangle but keeps four edges, while left-right reflection keeps four triangles and two edges. Printing the table checks that, and checks at the same time that every symmetry really does carry the hexagon onto itself.

⟨ Print the twelve symmetries 15 ⟩ ≡

```
hex := hexagon()
here := map[string]bool{}
for _, t := range hex {
    here[t.name()] = true
}
for _, s := range symmetries() {
    ⟨ Measure one symmetry and print its row 16 ⟩
}
```

This code is used in section 3.

16. The order is read off by iterating the symmetry on one triangle until it comes home. An edge is fixed when some triangle's edge in direction k is carried to that same edge.

⟨ Measure one symmetry and print its row 16 ⟩ ≡

```
onto, fixed, ord := true, 0, 1
for _, t := range hex {
    u := s.apply(t)
    if ¬here[u.name()] {
        onto = false
    }
    if u.name() ≡ t.name() {
        fixed++
    }
    for n, v := 1, s.apply(t); v.name() ≠ t.name(); n++ {
        v = s.apply(v)
        if n + 1 > ord {
            ord = n + 1
        }
    }
}
fe := map[string]bool{}
for _, t := range hex {
    for k := 0; k < 3; k++ {
        if s.apply(t).edges()[slot[s.perm[k]]] ≡ t.edges()[slot[k]] {
            fe[t.edges()[slot[k]]] = true
        }
    }
}
fmt.Printf("%-32s order=%d fixed triangles=%2d fixed edges=%2d onto=%v\n",
    s.name, ord, fixed, len(fe), onto)
```

This code is used in section 15.

17. Coordinate 0 is the slant that answer 126 writes with a slash, coordinate 1 the horizontal edge, coordinate 2 the other slant; so this table turns a coordinate into a position in *edges*( ).

⟨ Declarations 4 ⟩ +≡

```
var slot = [3]int{1, 0, 2}
```

18. The problem of answer 126. Twenty-four primary items for the triangles, twenty-four more for the tiles, forty-two secondary items for the edges, and $24 \cdot 64 = 1536$ options. Adding a primary item $*$ and one option that paints every boundary edge with colour a gives answer 126's problem exactly; leaving it out gives the free-border problem that exercise 129 is about.

Every model in this program starts with the same item line, so it is written once here. The caller supplies g, which renames an edge; all but one model lets it be the identity.

⟨ Write the item line 18 ⟩ ≡

```
var b strings.Builder
hex := hexagon()
for _, t := range hex {
    fmt.Fprintf(&b, "%s␣", t.name())
}
for _, tl := range tiles() {
    fmt.Fprintf(&b, "%s␣", tl)
}
⟨ Add the border item if it is wanted 19 ⟩
b.WriteString("|")
count := map[string]int{}
for _, t := range hex {
    for _, e := range t.edges() {
        count[g(e)]++
    }
}
var all, edge []string
for e, n := range count {
    all = append(all, e)
    if n ≡ 1 {
        edge = append(edge, e)
    }
}
sort.Strings(all)
sort.Strings(edge)
for _, e := range all {
    fmt.Fprintf(&b, "␣%s", e)
}
b.WriteString("\n")
⟨ Add the border option if it is wanted 20 ⟩
```

This code is used in sections 21, 28, 39, 47, 57.

19. ⟨ Add the border item if it is wanted 19 ⟩ ≡

```
if border {
    b.WriteString("*␣")
}
```

This code is used in section 18.

20. ⟨ Add the border option if it is wanted 20 ⟩ ≡

```
if border {
    b.WriteString("*")
    for _, e := range edge {
        fmt.Fprintf(&b, "␣%s:a", e)
    }
    b.WriteString("\n")
}
```

This code is used in section 18.

21. ⟨ Functions 5 ⟩ +≡

```
func base(border bool) string {
    g := func(e string) string { return e }
    ⟨ Write the item line 18 ⟩
    for _, t := range hex {
        e := t.edges()
        for i := 0; i < 64; i++ {
            c := triple(i)
            fmt.Fprintf(&b, "%s␣%s␣%s:%c␣%s:%c␣%s:%c\n",
                t.name(), canon(c), e[0], c[0], e[1], c[1], e[2], c[2])
        }
    }
    return b.String()
}
```

22. The 64 colourings of a triangle are indexed by a number in base four.

⟨ Functions 5 ⟩ +≡

```
func triple(i int) [3]byte {
    return [3]byte{byte('a' + i/16), byte('a' + i/4 % 4), byte('a' + i % 4)}
}
```

23. Only the pinned border is worth running: with the border free the same items have hundreds of millions of solutions for each of the two million admissible border patterns of exercise 127, and no useful number comes out of counting them all. What the free border does is make the symmetric families of exercise 129 much larger than answer 126's total, which is how one knows the border condition is not meant to carry over.

⟨ Reproduce answer 126 23 ⟩ ≡

```
n, nodes, el := solve(base(true))
fmt.Printf("%-32s␣%12d␣solutions␣%14d␣nodes␣%s\n",
    "border␣all␣white␣(answer␣126)", n, nodes, el)
```

This code is used in section 3.

24. Every mode ends by handing a problem to the solver and counting what comes back.

⟨ Functions 5 ⟩ +≡

```
func solve(in string)(int, uint64, time.Duration) {
    t0 := time.Now()
    xc := cells.NewXCC()
    n := 0
    for range xc.Dance(strings.NewReader(in)).Solutions {
        n++
    }
    return n, xc.Nodes(), time.Since(t0).Round(time.Millisecond)
}
```

25. Strong symmetry. A strong symmetry is a pair: a symmetry h of the hexagon and a permutation π of the colours, with h carrying the pattern to its π-recolouring. Since h and π must have the same order, and since a rotation of order 3 or 6 would need a colour permutation of that order—which on four colours must fix a colour, whose solid tile would then have to sit on a triangle the rotation fixes, and there are none—only involutions are worth testing.

There are ten involutions of four colours, counting the identity.

⟨ Functions 5 ⟩ +≡

```
func involutions() []map[byte]byte {
    cols := []byte{'a', 'b', 'c', 'd'}
    var out []map[byte]byte
    for i := 0; i < 24; i++ {
        p := map[byte]byte{}
        perm := []byte{cols[i/6], 0, 0, 0}
        ⟨ Build the permutation numbered i 26 ⟩
        ok := true
        for _, c := range cols {
            if p[p[c]] ≠ c {
                ok = false
            }
        }
        if ok ∧ ¬seenPerm(out, p) {
            out = append(out, p)
        }
    }
    return out
}
```

26. ⟨ Build the permutation numbered i 26 ⟩ ≡

```
rest := []byte{}
for _, c := range cols {
    if c ≠ perm[0] {
        rest = append(rest, c)
    }
}
perm[1] = rest[i/2 % 3]
rest2 := []byte{}
for _, c := range rest {
    if c ≠ perm[1] {
        rest2 = append(rest2, c)
    }
}
perm[2] = rest2[i % 2]
for _, c := range rest2 {
    if c ≠ perm[2] {
        perm[3] = c
    }
}
for k, c := range cols {
    p[c] = perm[k]
}
```

This code is used in section 25.

27. ⟨ Functions 5 ⟩ +≡

```
func seenPerm(out []map[byte]byte, p map[byte]byte) bool {
    for _, q := range out {
        same := true
        for _, c := range []byte { 'a', 'b', 'c', 'd' }{
            if q[c] ≠ p[c] {
                same = false
            }
        }
        if same {
            return true
        }
    }
    return false
}
```

28. The model pairs each triangle with its image: one option per orbit, colouring the representative freely and its image by the moved, recoloured triple. Three things have to be got right. A triangle the symmetry fixes must be its own image. A colouring whose two halves would ask for the same tile twice is no good. And when the two triangles of an orbit share an edge, the option must name that edge once, and only if the halves agree about its colour.

⟨ Functions 5 ⟩ +≡

```
func symModel(h sym, pi map[byte]byte)(string, int) {
    g := func(e string) string { return e }
    border := false
    ⟨ Write the item line 18 ⟩
    done := map[string]bool{}
    nopt := 0
    for _, t := range hex {
        u := h.apply(t)
        if done[t.name()] {
            continue
        }
        done[t.name()], done[u.name()] = true, true
        ⟨ Write the options for one orbit of h 29 ⟩
    }
    return b.String(), nopt
}
```

29. Coordinate k of the first triangle becomes coordinate $h.perm[k]$ of the second, so *slot* turns each into a position in $edges()$.

```
⟨ Write the options for one orbit of h 29 ⟩ ≡
    te, ue := t.edges(), u.edges()
    for i := 0; i < 64; i++ {
        c := triple(i)
        var d [3]byte
        for k := 0; k < 3; k++ {
            d[slot[h.perm[k]]] = pi[c[slot[k]]]
        }
        if t.name() ≡ u.name() {
            ⟨ Write the option for a triangle the symmetry fixes 30 ⟩
            continue
        }
        if canon(c) ≡ canon(d) {
            continue
        }
        ⟨ Write the paired option, merging any shared edge 31 ⟩
    }
```

This code is used in section 28.

30.

```
⟨ Write the option for a triangle the symmetry fixes 30 ⟩ ≡
    if c ≠ d {
        continue
    }
    fmt.Fprintf(&b, "%s␣%s␣%s:%c␣%s:%c␣%s:%c\n",
        t.name(), canon(c), te[0], c[0], te[1], c[1], te[2], c[2])
    nopt++
```

This code is used in section 29.

31.

```
⟨ Write the paired option, merging any shared edge 31 ⟩ ≡
    col := map[string]byte{}
    clash := false
    for k := 0; k < 3; k++ {
        for _, p := range [2]struct {
            e string
            c byte
        }{{te[k], c[k]}, {ue[k], d[k]}}{
            if old, ok := col[p.e]; ok ∧ old ≠ p.c {
                clash = true
            }
            col[p.e] = p.c
        }
    }
    if clash {
        continue
    }
    ⟨ Print the merged option 32 ⟩
```

This code is used in section 29.

32. ⟨ Print the merged option 32 ⟩ ≡

```
fmt.Fprintf(&b, "%s␣%s␣%s␣%s", t.name(), canon(c), u.name(), canon(d))
var es []string
for e := range col {
    es = append(es, e)
}
sort.Strings(es)
for _, e := range es {
    fmt.Fprintf(&b, "␣%s:%c", e, col[e])
}
b.WriteString("\n")
nopt++
```

This code is used in section 31.

33. Sweeping every reflection against every involution takes a few seconds and settles the answer's claim that strong reflection symmetry is impossible. The half-turn is swept too, to see that only the fixed-point-free involutions survive—which is what licenses the answer's "assume that rotation changes $a \leftrightarrow d$, $b \leftrightarrow c$".

⟨ Ask whether a strong symmetry is possible 33 ⟩ ≡

```
for _, h := range symmetries() {
    if ¬strings.Contains(h.name, "reflection") ∧ h.name ≠ halfTurn {
        continue
    }
    for _, pi := range involutions() {
        ⟨ Try one symmetry against one involution 34 ⟩
    }
}
```

This code is used in section 3.

34. The half-turn against a fixed-point-free involution is the one case that has solutions, and there are 68 million of them; that is the subject of the next section, so here it is only named.

⟨ Try one symmetry against one involution 34 ⟩ ≡

```
in, nopt := symModel(h, pi)
fixed := 0
for _, c := range []byte { 'a', 'b', 'c', 'd' }{
    if pi[c] ≡ c {
        fixed++
    }
}
if h.name ≡ halfTurn ∧ fixed ≡ 0 {
    fmt.Printf("%-30s␣%s␣%5d␣options␣(counted␣separately)\n",
        h.name, piName(pi), nopt)
    continue
}
n := 0
if nopt > 0 {
    n, _, _ = solve(in)
}
fmt.Printf("%-30s␣%s␣%5d␣options␣%8d␣solutions\n",
    h.name, piName(pi), nopt, n)
```

This code is used in section 33.

35. ⟨ Declarations 4 ⟩ +≡

```
const halfTurn = "[0␣1␣2]␣flip=true␣rotation"
```

36. ⟨ Functions 5 ⟩ +≡

```
func piName(pi map[byte]byte) string {
    s := ""
    for _, c := range []byte { 'a', 'b', 'c', 'd' }{
        s += string(pi[c])
    }
    return s
}
```

37. The paired options. For the half-turn the answer fixes the colour involution and writes out the 768 paired options directly. The rotation sends the upward triangle (x, y) to the downward triangle $(3-x, 3-y)'$, and correspondingly moves the edges.

⟨ Functions 5 ⟩ +≡

```
func rotTri(t tri) tri { return tri{3 − t.x, 3 − t.y, ¬t.down} }
```

38. ⟨ Declarations 4 ⟩ +≡

```
var swap = map[byte]byte{'a': 'd', 'd': 'a', 'b': 'c', 'c': 'b'}
```

39. Here the caller may ask for the edges to be glued, which is how the plane-tiling proviso gets imposed; otherwise g is the identity again.

⟨ Functions 5 ⟩ +≡

```
func strongModel(tiling bool)(string, int) {
    g := func(e string) string { return e }
    if tiling {
        g = glue
    }
    border := false
    ⟨ Write the item line 18 ⟩
    nopt := 0
    for _, t := range hex {
        if t.down {
            continue
        }
        u := rotTri(t)
        ⟨ Write the two halves of one rotated pair 40 ⟩
    }
    return b.String(), nopt
}
```

40. ⟨ Write the two halves of one rotated pair 40 ⟩ ≡

```
te, ue := t.edges(), u.edges()
for i := 0; i < 64; i++ {
    c := triple(i)
    d := [3]byte{swap[c[0]], swap[c[1]], swap[c[2]]}
    if canon(c) ≡ canon(d) {
        continue
    }
    fmt.Fprintf(&b, "%s␣%s␣%s:%c␣%s:%c␣%s:%c␣%s␣%s␣%s:%c␣%s:%c␣%s:%c\n",
        t.name(), canon(c), g(te[0]), c[0], g(te[1]), c[1], g(te[2]), c[2],
        u.name(), canon(d), g(ue[0]), d[0], g(ue[1]), d[1], g(ue[2]), d[2])
    nopt++
}
```

This code is used in section 39.

41. An orbit of strongly symmetric placements holds 144 of them, and its three fixed-point-free involutions share those equally, so 48 of them are strong for the one this model pins down.

⟨ Count the strongly symmetric placements 41 ⟩ ≡

```
in, nopt := strongModel(false)
n, nodes, el := solve(in)
fmt.Printf("%d paired options, %d solutions, %d nodes, %s\n", nopt, n, nodes, el)
fmt.Printf("essentially distinct = %d/48 = %d remainder %d (answer 129 says 1417168)\n",
    n, n/48, n % 48)
```

This code is used in section 3.

42. Tiling the plane. The bracketed remark in answer 129 notices that its illustrated pattern tiles the plane by translation alone, and counts how many essentially distinct solutions do. The condition is that opposite boundary edges agree; naming four of the six pairs, the answer leaves the rest to the reader.

⟨ Declarations 4 ⟩ +≡

```
var opposite = map[string]string{
    "-04": "-20", "-14": "-30",
    "/02": "/40", "/03": "/41",
    `\01`: `\23`, `\10`: `\32`,
}
```

43. Renaming each edge to the representative of its pair turns the condition into plain item sharing: two edges that must agree become one secondary item.

⟨ Functions 5 ⟩ +≡

```
func glue(e string) string {
    if r, ok := opposite[e]; ok {
        return r
    }
    return e
}
```

44. ⟨ Count the ones that tile the plane 44 ⟩ ≡

```
in, nopt := strongModel(true)
n, nodes, el := solve(in)
fmt.Printf("%d paired options, %d solutions, %d nodes, %s\n", nopt, n, nodes, el)
fmt.Printf("essentially distinct = %d/48 = %d remainder %d (answer 129 says 40208)\n",
    n, n/48, n % 48)
```

This code is used in section 3.

45. Colour patches. The colours inside a triangle meet at its three corners, and a corner is a boundary when the two edges beside it differ. Answer 129 carries that three-bit code on a new secondary item. Indexing the bits by coordinate rather than by edge—corner 0 is where the edges of coordinates 1 and 2 meet—makes a symmetry permute the bits exactly as it permutes the coordinates.

⟨ Functions 5 ⟩ +≡

```
func cmask(c [3]byte) [3]bool {
    return [3]bool{c[0] ≠ c[2], c[1] ≠ c[2], c[0] ≠ c[1]}
}
```

46. ⟨ Functions 5 ⟩ +≡

```
func packm(m [3]bool) int {
    v := 0
    for i, s := range m {
        if s {
            v |= 1 ≪ i
        }
    }
    return v
}
```

47. Weak symmetry. The condition weak symmetry under g imposes is

$$\mathrm{mask}(t)[k] = \mathrm{mask}(g(t))[g.\mathrm{perm}[k]]$$

for every triangle t and every corner k. So one secondary item per orbit suffices, provided each triangle reports its own mask read through the symmetry that brought it there.

Answer 129 introduces its items “for each triangle (x, y) or $(x, y)'$ with $y > 1$”, which is one per orbit of the half-turn. Doing the same for a group in general is what this section is for.

⟨ Functions 5 ⟩ +≡

```
func weakGroup(gs []sym)(string, int) {
    g := func(e string) string { return e }
    border := false
    ⟨ Write the item line 18 ⟩
    ⟨ Choose an orbit representative for each triangle 48 ⟩
    ⟨ Add one patch item per orbit 49 ⟩
    nopt := 0
    for _, t := range hex {
        ⟨ Write the options for one triangle 50 ⟩
    }
    return b.String(), nopt
}
```

48. ⟨ Choose an orbit representative for each triangle 48 ⟩ ≡

```
rep := map[string]string{}
byName := map[string]tri{}
for _, t := range hex {
    byName[t.name()] = t
}
for _, t := range hex {
    if _, ok := rep[t.name()]; ok {
        continue
    }
    rep[t.name()] = t.name()
    for _, s := range gs {
        if u := s.apply(t); rep[u.name()] == "" {
            rep[u.name()] = t.name()
        }
    }
}
```

This code is used in sections 47, 57.

49. The item line was already written, so the patch items are appended to it before any option goes out. They are secondary, and they carry the mask as a colour.

⟨ Add one patch item per orbit 49 ⟩ ≡

```
head, rest, _ := strings.Cut(b.String(), "\n")
done := map[string]bool{}
for _, t := range hex {
    if r := rep[t.name()]; ¬done[r] {
        done[r] = true
        head += " w" + r
    }
}
b.Reset()
b.WriteString(head + "\n" + rest)
```

This code is used in sections 47, 57.

50. A triangle may be reachable from its representative by more than one symmetry—this is exactly what happens to the four triangles a left-right reflection fixes—and then every one of them has to give the same answer. Leaving that out is the one mistake this program was written to avoid: a fixed triangle would otherwise be an orbit of one, its item would appear in only its own options, and the reflection's effect on its three corners would go unsaid.

⟨ Write the options for one triangle 50 ⟩ ≡

```
r := byName[rep[t.name()]]
var ways [][3]int
for _, s := range gs {
    if s.apply(r).name() ≡ t.name() {
        ways = append(ways, s.perm)
    }
}
e := t.edges()
for i := 0; i < 64; i++ {
    c := triple(i)
    ⟨ Skip the option if it breaks the solid-tile rule 64 ⟩
    ⟨ Work out the patch colour, or skip the option 51 ⟩
    fmt.Fprintf(&b, "%s %s%s:%c %s:%c %s:%c w%s:%d\n",
        t.name(), canon(c), e[0], c[0], e[1], c[1], e[2], c[2],
        rep[t.name()], v)
    nopt++
}
```

This code is used in section 47.

51. ⟨ Work out the patch colour, or skip the option 51 ⟩ ≡

```
m := cmask(c)
v, ok := −1, true
for _, p := range ways {
    var n [3]bool
    for k := 0; k < 3; k++ {
        n[k] = m[p[k]]
    }
    if w := packm(n); v < 0 {
        v = w
    } else if w ≠ v {
        ok = false
    }
}
if ¬ok {
    continue
}
```

This code is used in section 50.

52. The *dump* flag prints the problem rather than solving it, which is how one compares two models that ought to be the same.

53. Four groups are worth counting. Under the half-turn, which is central, the raw count is the whole of every orbit that is weakly symmetric. Under a reflection it is 96 placements per weak-but-not-strong class and 48 per strong one. Under the group generated by a left-right reflection and the half-turn it is the same, counting the classes that have both kinds of weak reflection symmetry.

⟨ Count the weakly symmetric placements 53 ⟩ ≡

```
gs := []sym{}
for _, s := range symmetries() {
    if s.name ≡ "[0 1 2] flip=false rotation" {
        gs = append(gs, s)
    }
}
⟨ Add the symmetries the group needs 54 ⟩
⟨ Set the solid-tile rule, if one was asked for 65 ⟩
in, nopt := weakGroup(gs)
if *dump {
    fmt.Print(in)
    return
}
if *check {
    ⟨ Count the solutions, rebuilding every one's patches 74 ⟩
    return
}
n, nodes, el := solve(in)
fmt.Printf("%s: %d options, %d solutions, %d nodes, %s\n", *group, nopt, n, nodes, el)
⟨ Say what the answer predicts 55 ⟩
```

This code is used in section 3.

54. ⟨ Add the symmetries the group needs 54 ⟩ ≡

```
for _, s := range symmetries() {
    switch *group {
    case "rot":
        if s.name ≡ halfTurn {
            gs = append(gs, s)
        }
    case "top":
        if s.name ≡ "[0 2 1] flip=true reflection" {
            gs = append(gs, s)
        }
    case "left":
        if s.name ≡ "[0 2 1] flip=false reflection" {
            gs = append(gs, s)
        }
    case "both":
        if s.perm ≡ [3]int { 0, 2, 1 } ∨ s.name ≡ halfTurn{
            gs = append(gs, s)
        }
    case "s3":
        if ¬s.flip {
            gs = append(gs, s)
        }
    case "rot3":
        if ¬s.flip ∧ ¬strings.Contains(s.name, "reflection") {
            gs = append(gs, s)
        }
    }
}
```

This code is used in section 53.

55. ⟨ Say what the answer predicts 55 ⟩ ≡

```
switch *group {
case "rot":
    fmt.Printf("  144*1417168 + 288*609203 = %d\n", 144 * 1417168 + 288 * 609203)
case "top":
    fmt.Printf("  96*41608 + 48*261 = %d\n", 96 * 41608 + 48 * 261)
case "left":
    fmt.Printf("  answer 129 predicts 96*281618 + 48*261 = %d\n",
        96 * 281618 + 48 * 261)
    fmt.Printf("  this reading says   96*294457 + 48*261 = %d\n",
        96 * 294457 + 48 * 261)
case "both":
    fmt.Printf("  96*194 + 48*261 = %d\n", 96 * 194 + 48 * 261)
case "s3", "rot3":
    fmt.Println("  a class weak under these would count 288 rather than 96")
}
```

This code is used in section 53.

56. The special placements. Some placements are strongly symmetric under the half-turn and weakly symmetric under a reflection as well; the answer calls them special and counts $88 + 98 + 75 = 261$ of them. Putting the patch items of a reflection on top of the paired options counts them directly. A special class contributes $4 \cdot 8/2 = 16$ placements: four symmetries of the hexagon commute with the reflection, eight colour permutations carry the class's own involution to the pinned one, and the stabilizer of order two halves the result.

```
⟨ Count the special placements 56 ⟩ ≡
    for _, tau := range symmetries() {
        if ¬strings.Contains(tau.name, "reflection") ∨ ¬tau.flip {
            continue
        }
        in, nopt := specialModel(tau)
        n, nodes, el := solve(in)
        fmt.Printf("%-30s %4d options %8d solutions %10d nodes %s\n",
            tau.name, nopt, n, nodes, el)
        fmt.Printf("   %d/16 = %d remainder %d (answer 129 says 261)\n",
            n, n/16, n % 16)
    }
```

This code is used in section 3.

57. Both halves of a paired option must report a patch colour, and when the reflection happens to put them in the same orbit the two reports have to agree.

```
⟨ Functions 5 ⟩ +≡
    func specialModel(tau sym)(string, int) {
        g := func(e string) string { return e }
        border := false
        ⟨ Write the item line 18 ⟩
        gs := []sym{{[3]int{0, 1, 2}, false, ""}, tau}
        ⟨ Choose an orbit representative for each triangle 48 ⟩
        ⟨ Add one patch item per orbit 49 ⟩
        nopt := 0
        for _, t := range hex {
            if t.down {
                continue
            }
            u := rotTri(t)
            ⟨ Write one special pair 58 ⟩
        }
        return b.String(), nopt
    }
```

58. ⟨ Write one special pair 58 ⟩ ≡

```
te, ue := t.edges(), u.edges()
for i := 0; i < 64; i++ {
    c := triple(i)
    d := [3]byte{swap[c[0]], swap[c[1]], swap[c[2]]}
    if canon(c) ≡ canon(d) {
        continue
    }
    ct, cu := patchOf(t, c, rep, byName, gs), patchOf(u, d, rep, byName, gs)
    if ct < 0 ∨ cu < 0 {
        continue
    }
    if rep[t.name()] ≡ rep[u.name()] ∧ ct ≠ cu {
        continue
    }
    ⟨ Print the two halves and their patch items 59 ⟩
}
```

This code is used in section 57.

59. ⟨ Print the two halves and their patch items 59 ⟩ ≡

```
fmt.Fprintf(&b, "%s␣%s␣%s:%c␣%s:%c␣%s:%c␣%s␣%s␣%s:%c␣%s:%c␣%s:%c␣w%s:%d",
    t.name(), canon(c), te[0], c[0], te[1], c[1], te[2], c[2],
    u.name(), canon(d), ue[0], d[0], ue[1], d[1], ue[2], d[2],
    rep[t.name()], ct)
if rep[t.name()] ≠ rep[u.name()] {
    fmt.Fprintf(&b, "␣w%s:%d", rep[u.name()], cu)
}
b.WriteString("\n")
nopt++
```

This code is used in section 58.

60. The patch colour of a triangle, or −1 when the symmetries that reach it disagree and the colouring has to be dropped.

⟨ Functions 5 ⟩ +≡

```
func patchOf(t tri, c [3]byte, rep map[string]string,
    byName map[string]tri, gs []sym) int {
    m := cmask(c)
    v := −1
    for _, s := range gs {
        if s.apply(byName[rep[t.name()]]).name() ≠ t.name() {
            continue
        }
        var n [3]bool
        for k := 0; k < 3; k++ {
            n[k] = m[s.perm[k]]
        }
        if w := packm(n); v < 0 {
            v = w
        } else if w ≠ v {
            return −1
        }
    }
    return v
}
```

61. Splitting the left-right count. The identity for a left-right reflection is the one that would not come out. Two of the three reflections, run separately, both gave 28,280,400, and answer 129 predicts $96 \cdot 281618 + 48 \cdot 261 = 27{,}047{,}856$. The gap is $1{,}232{,}544 = 96 \cdot 12839$: a whole number of classes, which is the shape the bridge of the first sections demands, so it is not a rounding of anything.

The introduction says this program cannot check the answer's six types one at a time, because they are a bookkeeping device of its own. That is true of the types themselves, but the family can still be cut into pieces the answer's case analysis has to respect, and the cut it offers is the one it makes itself: where the four single-colour tiles sit. A reflection carries solid triangles to solid triangles, so the four of them are permuted; answer 129 splits the left-right case according to whether any of them is fixed.

62. A reflection of the left-right kind fixes four triangles and moves the other twenty in pairs. Both lists are wanted: the fixed ones say which of the answer's two branches a placement is in, and the pairs give the finer split within the second branch.

⟨ Functions 5 ⟩ +≡

```
func split(s sym)(fixed []string, orb [][2]string) {
    seen := map[string]bool{}
    for _, t := range hexagon() {
        u := s.apply(t)
        if u.name() ≡ t.name() {
            fixed = append(fixed, t.name())
            continue
        }
        if seen[t.name()] {
            continue
        }
        seen[t.name()], seen[u.name()] = true, true
        orb = append(orb, [2]string{t.name(), u.name()})
    }
    return
}
```

63. A rule says, for some of the triangles, whether the tile placed there must be a solid one. Triangles the rule says nothing about are free.

⟨ Declarations 4 ⟩ +≡

```
var solidRule map[string]bool
```

64. ⟨ Skip the option if it breaks the solid-tile rule 64 ⟩ ≡

```
if want, ok := solidRule[t.name()]; ok {
    if want ≠ (c[0] ≡ c[1] ∧ c[1] ≡ c[2]) {
        continue
    }
}
```

This code is used in section 50.

65. Two ways of asking. With *−fixed* the rule names only the four fixed triangles, so the sixteen values of the mask cut the whole left-right family into sixteen disjoint pieces; the eight with an odd number of bits must come out empty, since the solid tiles that are not fixed pair up and so an even number of them is fixed. With *−conf* the rule covers every triangle: the two named orbits carry solid tiles and nobody else does, which cuts the branch where no solid tile is fixed into $\binom{10}{2} = 45$ pieces.

⟨ Set the solid-tile rule, if one was asked for 65 ⟩ ≡

```
if *fixedMask ≥ 0 ∨ *conf ≠ "" {
    var tau sym
    for _, t := range symmetries() {
        if t.name ≡ leftRight {
            tau = t
        }
    }
    fixed, orb := split(tau)
    solidRule = map[string]bool{}
    ⟨ Fill in the rule 66 ⟩
}
```

This code is used in section 53.

66. ⟨ Fill in the rule 66 ⟩ ≡

```
if *fixedMask ≥ 0 {
    for i, n := range fixed {
        solidRule[n] = *fixedMask ≫ i & 1 ≡ 1
    }
    fmt.Printf("fixed triangles %v, mask %d\n", fixed, *fixedMask)
} else {
    var i, j int
    fmt.Sscanf(*conf, "%d,%d", &i, &j)
    for _, t := range hexagon() {
        solidRule[t.name()] = false
    }
    for _, k := range []int { i, j }{
        solidRule[orb[k][0]], solidRule[orb[k][1]] = true, true
    }
    fmt.Printf("solid tiles on orbits %v and %v\n", orb[i], orb[j])
}
```

This code is used in section 65.

67. ⟨ Declarations 4 ⟩ +≡

```
const leftRight = "[0 2 1] flip=false reflection"
```

68. Checking the patches from scratch. A count is only as good as the model that produced it, and the model above says “weakly symmetric” in terms of three-bit patch items. So there is a second opinion here that shares nothing with it: each solution is turned back into a colouring of the 42 edges, its colour patches are rebuilt by union-find, and the reflection is asked directly whether it carries that partition to itself.

The same pass takes a census. For a placement p let s be the order of its stabilizer and N the number of g in the hexagon group with $g^{-1}\tau g$ weak for p. The bridge says the class of p meets the model in $24N/s$ placements, so summing $s/(24N)$ over all solutions counts classes—with no multiplier taken on faith.

⟨ Declarations 4 ⟩ +≡

```
var edgeIdx map[string]int
var edgeList []string
```

69. ⟨ Functions 5 ⟩ +≡

```
func setupEdges() {
    edgeIdx = map[string]int{}
    seen := map[string]bool{}
    for _, t := range hexagon() {
        for _, e := range t.edges() {
            if ¬seen[e] {
                seen[e] = true
                edgeList = append(edgeList, e)
            }
        }
    }
    sort.Strings(edgeList)
    for i, e := range edgeList {
        edgeIdx[e] = i
    }
}
```

70. Coordinate $slot[k]$ of a triangle owns its edge k, and a symmetry sends coordinate k to coordinate $perm[k]$, so it sends edge k of t to edge $slot[perm[slot[k]]]$ of its image. An interior edge belongs to two triangles and must be sent to the same place by both, which is worth checking.

⟨ Functions 5 ⟩ +≡

```
func edgeMap(s sym) []int {
    m := make([]int, len(edgeList))
    for i := range m {
        m[i] = -1
    }
    for _, t := range hexagon() {
        u := s.apply(t)
        for k := 0; k < 3; k++ {
            a := edgeIdx[t.edges()[k]]
            b := edgeIdx[u.edges()[slot[s.perm[slot[k]]]]]
            if m[a] ≥ 0 ∧ m[a] ≠ b {
                panic("the two triangles of an edge disagree")
            }
            m[a] = b
        }
    }
    return m
}
```

71. ⟨ Declarations 4 ⟩ +≡

```
var root [42]int
```

72. ⟨ Functions 5 ⟩ +≡

```
func find(x int) int {
    for root[x] ≠ x {
        root[x] = root[root[x]]
        x = root[x]
    }
    return x
}
```

73. The composition of two symmetries, which the census needs to name the conjugate $g^{-1}\tau g$ without ever inverting anything: it is the u with $gu = \tau g$.

⟨ Functions 5 ⟩ +≡

```
func compose(a, b sym) sym {
    var p [3]int
    for i := 0; i < 3; i++ {
        p[i] = a.perm[b.perm[i]]
    }
    for _, s := range symmetries() {
        if s.perm ≡ p ∧ s.flip ≡ (a.flip ≠ b.flip) {
            return s
        }
    }
    panic("not a symmetry")
}
```

74. ⟨ Count the solutions, rebuilding every one's patches 74 ⟩ ≡

```
setupEdges()
all := symmetries()
var tau sym
for _, s := range all {
    if s.name ≡ leftRight {
        tau = s
    }
}
ems := make([][]int, len(all))
for i, s := range all {
    ems[i] = edgeMap(s)
}
```

⟨ Name the conjugate of the reflection by each symmetry 75 ⟩

⟨ Run through the solutions and take the census 76 ⟩

This code is used in section 53.

75. ⟨ Name the conjugate of the reflection by each symmetry 75 ⟩ ≡

```
conj := make([]int, len(all))
for j, g := range all {
    want := compose(tau, g)
    for i, u := range all {
        if compose(g, u).name ≡ want.name {
            conj[j] = i
        }
    }
}
```

This code is used in section 74.

76. ⟨ Run through the solutions and take the census 76 ⟩ ≡

```
t0 := time.Now()
xc := cells.NewXCC()
var col [42]byte
var num, raw, bad int64
tally := map[[2]int]int64{}
for sol := range xc.Dance(strings.NewReader(in)).Solutions {
    ⟨ Read the colours off the solution 77 ⟩
    ⟨ Rebuild the colour patches 78 ⟩
    ⟨ Ask every symmetry whether it is weak, and whether it is strong 79 ⟩
    ⟨ Add the solution to the census 82 ⟩
}
fmt.Printf("%d␣solutions,␣%d␣of␣them␣not␣really␣weak,␣%s\n",
    raw, bad, time.Since(t0).Round(time.Second))
fmt.Printf("classes␣=␣%d/288␣=␣%g\n", num, float64(num)/288)
for k, v := range tally {
    fmt.Printf("␣␣N=%d␣s=%d:␣%d␣placements,␣%g␣classes\n",
        k[0], k[1], v, float64(v) * float64(k[1])/float64(24 * k[0]))
}
```

This code is used in section 74.

77. ⟨ Read the colours off the solution 77 ⟩ ≡

```
for _, opt := range sol {
    for _, w := range opt {
        k := strings.IndexByte(w, ':')
        if k < 0 {
            continue
        }
        if i, ok := edgeIdx[w[:k]]; ok {
            col[i] = w[k + 1]
        }
    }
}
```

This code is used in section 76.

78. Two edges of a triangle lie in the same patch exactly when their colours agree, and the patches of the whole hexagon are what those merges generate.

⟨ Rebuild the colour patches 78 ⟩ ≡

```
for i := range root {
    root[i] = i
}
for _, t := range hexagon() {
    e := t.edges()
    for i := 0; i < 3; i++ {
        for j := i + 1; j < 3; j++ {
            if col[edgeIdx[e[i]]] ≡ col[edgeIdx[e[j]]] {
                root[find(edgeIdx[e[i]])] = find(edgeIdx[e[j]])
            }
        }
    }
}
```

This code is used in section 76.

79. A symmetry is weak when the permutation it makes of the edges carries the partition to itself, which is to say that the induced map on patches is well defined both ways round. It is strong when the colouring itself comes back recoloured, which is a permutation of the four colours read off edge by edge.

⟨ Ask every symmetry whether it is weak, and whether it is strong 79 ⟩ ≡

```
var weak, strong [12]bool
for i := range all {
    ⟨ Ask whether symmetry i is weak 80 ⟩
    ⟨ Ask whether symmetry i is strong 81 ⟩
}
```

This code is used in section 76.

80. ⟨ Ask whether symmetry i is weak 80 ⟩ ≡

```
var to, from [42]int
for e := range to {
    to[e], from[e] = −1, −1
}
weak[i] = true
for e := 0; e < 42; e++ {
    r, q := find(e), find(ems[i][e])
    if to[r] < 0 {
        to[r] = q
    } else if to[r] ≠ q {
        weak[i] = false
    }
    if from[q] < 0 {
        from[q] = r
    } else if from[q] ≠ r {
        weak[i] = false
    }
}
```

This code is used in section 79.

81. ⟨ Ask whether symmetry i is strong 81 ⟩ ≡

```
var pi, rev [4]byte
for k := range pi {
    pi[k], rev[k] = 255, 255
}
strong[i] = true
for e := 0; e < 42; e++ {
    c, d := col[e] - 'a', col[ems[i][e]] - 'a'
    if (pi[c] ≠ 255 ∧ pi[c] ≠ d) ∨ (rev[d] ≠ 255 ∧ rev[d] ≠ c) {
        strong[i] = false
    }
    pi[c], rev[d] = d, c
}
```

This code is used in section 79.

82. ⟨ Add the solution to the census 82 ⟩ ≡

```
if ¬weak[idxOf(all, tau)] {
    bad++
    continue
}
s, n := 0, 0
for i := range all {
    if strong[i] {
        s++
    }
    if weak[conj[i]] {
        n++
    }
}
raw++
num += int64(12 * s/n)
tally[[2]int{n, s}]++
```

This code is used in section 76.

83. ⟨ Functions 5 ⟩ +≡

```
func idxOf(all []sym, s sym) int {
    for i, t := range all {
        if t.name ≡ s.name {
            return i
        }
    }
    return -1
}
```

84. Index.

Symmetrical Hexagons